

ChatGPT - ZaloAI025

<https://github.com/roboflow/rf-detr>

đọc code này, tôi muốn chỉnh sửa chút:

1. theo tôi biết thì model này đang train bbox + classification -> tôi muốn đổi lại architecture một chút : chỉ train với input là reference images -> vẽ bbox -> không cần label nào
2. chỉ ra chỗ nào là kiến trúc của detr và cho tôi vài phương pháp sửa theo hướng của tôi

Ngắt mạch theo đúng repo:

1. Thay "detection (cls+bbox)" → "reference-conditioned, box-only"

- Ràng buộc hiện tại: RF-DETR gọi `Model(**config.dict())` từ lớp bao `RFDETR` trong `rfdetr/detr.py`. Đây là nơi dựng mô hình và nơi hàm `train()` được gọi.

`rfdetr.roboflow.c...`

- Cắt bỏ phân loại: giữ nguyên pipeline backbone → encoder/decoder → query boxes, nhưng loại bỏ nhánh classification và toàn bộ loss liên quan đến lớp. Tối thiểu cần:
 - Trả `pred_boxes` bình thường.
 - Ép `pred_logits` thành dummy (hoặc hằng 0) và không tính loss cls.
 - Hungarian matcher chạy "box-only" với $\text{cost} = \alpha \cdot L1 + \beta \cdot \text{GloU}$ (không dùng lớp).

Sửa kỹ thuật (đường đi cụ thể):

- Điểm vào: `RFDETR.get_model()` trả `Model(...)` trong `rfdetr/detr.py`. Chỉnh bên trong lớp `Model` để:
 - Bỏ/khóa nhánh `class_head` trong `forward` và outputs. `rfdetr.roboflow.c...`
 - Trong phần "criterion/loss", đặt weight của cross-entropy = 0, chỉ giữ L1 + GloU.
- Huỷ phụ thuộc vào nhãn:
 - Trong dataloader/targets: set `labels` = 0 hoặc bỏ hẳn field; matcher và criterion phải không đọc `labels`.

Ví dụ patch tối thiểu (ý tưởng, áp vào các vị trí tương đương trong `Model.forward` và `criterion`):

python

```
# --- Model.forward(...)
out = {}
out["pred_boxes"] = box_head(hs)          # như cũ
out["pred_logits"] = torch.zeros((*hs.shape[:2], 1), device=hs.device) #
dummy
return out
```

python

```
# --- criterion.compute(...)
loss_bbox = l1(pred_boxes, tgt_boxes) + lambda_giou * (1 - giou(pred_boxes,
tgt_boxes))
losses = {"loss_bbox": loss_bbox}
# bỏ: loss_ce / focal / cls
return losses
```

python

```
# --- matcher(costs)
# cost_class = 0
cost_bbox = lam_l1 * l1_cost(pred_boxes, tgt_boxes) + lam_giou *
giou_cost(pred_boxes, tgt_boxes)
# hungarian on cost_bbox
```

2. Chỉ ra "kiến trúc DETR" trong repo và cách sửa theo hướng "reference → bbox, no labels"

- Lớp bao & huấn luyện: `rfdetr/detr.py` (RFDETR, RFDETRMedium/Small/Nano kế thừa). `RFDETR.train()` gom config, callback, rồi gọi `self.model.train(...)`. Đây là cửa vào để chèn config "box-only", tắt W&B, early stopping, v.v. `rfdetr.roboflow.c... +1`
- Nơi dựng mô hình: `RFDETR.get_model(config) → Model(**config.dict())` trong `rfdetr/detr.py`. Đây là chỗ liên kết sang lớp `Model` chứa backbone, transformer, query embeddings, heads. Bạn chỉnh tại đây hoặc trong định nghĩa `Model`. `rfdetr.roboflow.c...`
- Tài liệu API cho các biến thể (Medium/Nano/Small) xác nhận pattern trên—các lớp size chỉ override `get_model_config`, còn core vẫn ở `rfdetr/detr.py`. `rfdetr.roboflow.c...`

Ba phương án để biến thành "reference-only, box-only":

A) Class-agnostic DETR + box-only loss (nhANH, ít rủi ro)

- Giữ nguyên kiến trúc, đặt `num_classes=1` trong `ModelConfig`, nhưng tắt hoàn toàn loss cls như trên (weight=0).
- Ở hậu xử lý, bỏ dùng score từ `pred_logits`; thay bằng:
 - (1) IoU-objectness: dùng $1 - \min(\text{cost_bbox})$ như "điểm tin cậy", hoặc
 - (2) top-K queries cố định mỗi ảnh.
- Dữ liệu: vì "không cần label", bạn phải tạo pseudo-boxes. Với "reference images" sẵn có, sinh pseudo box bằng:
 - Max-sim heatmap giữa feature map khung cảnh và embedding ảnh reference (CLIP/DINOv2), lấy connected component lớn nhất $\rightarrow$ box. Lặp lại qua augmentations để tăng độ ổn định. Huấn luyện với L1+GloU chỉ dùng pseudo box.
- Ưu: ít code đổi trong core; tận dụng pretrain RF-DETR. Nhược: độ chính xác phụ thuộc pseudo.

B) Reference-conditioned queries (chuẩn "matching")

- Thêm một "reference encoder" (chia sẻ backbone) để lấy feature `F_ref`. Khởi tạo object queries bằng cách chiếu `F_ref` sang không gian queries: `q0 = MLP( GAP(F_ref) )` hoặc dùng cross-attention (keys/values = `F_ref`, queries = learnable + ref token).
- Sửa `Model.forward(ref_img, frame_img)`:
 - Chạy backbone cho cả ref và frame.
 - Trước decoder, trộn `q0` với learnable queries, hoặc thêm 1–4 ref tokens.
- Matcher/loss như A (box-only). Pseudo box có thể lấy đơn giản: box phủ toàn bộ reference (nếu ref đã cropped object), hoặc heatmap tương tự A.
- Ưu: dùng thông tin tham chiếu trực tiếp; hợp bài toán "tìm đồ giống ref".

C) Teacher-free self-training từ reference crops

- Dùng chính ảnh reference (đã là crop) $\rightarrow$ coi box target = full image box. Huấn luyện đầu kỳ vài epoch để ổn định regression head; sau đó chạy trên video/frame thật, lấy top-1 box làm pseudo, refine lặp.
- Đơn giản nhất khi reference đã cắt tight bbox.

Thay đổi cụ thể cần làm trong code (tối thiểu để chạy):

1. Tạo "box-only criterion"

- Trong lớp criterion hiện tại (nơi tính CE/Focal + L1 + GloU), đặt weight cls = 0, hoặc xóa block tính CE. Kết quả chỉ trả `loss_bbox`, `loss_giou`.
- Xóa/đặt mặc định `target["labels"]`.

2. Hungarian matcher "box-only"

- Loại bỏ thành phần `cost_class`, chỉ giữ `cost_bbox = \lambda_1 * L1 + \lambda_2 * GIoU`.
- 3. Bật nhánh phân loại**
- Trong `Model.forward`, bỏ `class_head(hs)`; gán `pred_logits = zeros` cho tương thích API.
- 4. Hậu xử lý không dựa vào cls**
- Nếu pipeline của repo đòi `scores`, sinh `scores = sigmoid(-cost_bbox_min)` hoặc set `scores = 1.0` và lọc theo NMS/loU nội bộ của bạn.
- 5. Reference-conditioning (nếu chọn B)**
- Thêm `ref_img` vào `predict()/train()` của `RFDETR` trong `rfdetr/detr.py`.
 - Trong `Model`, thêm nhánh encode ref + module trộn vào queries (linear 或 multihead cross-attn nhẹ trước decoder).
 - Khi không có ref, fallback sang learnable queries để tương thích checkpoint pretrain.

Nơi gắn móc trong repo (theo tài liệu chính thức):

- `rfdetr/detr.py`: lớp `RFDETR`, `get_model`, `train`, `predict` — cửa vào để truyền thêm `ref_img` và cấu hình mới. rfdetr.roboflow.c... +1
- `Model(**config)`: nơi tổ chức backbone/transformer/heads — chỉnh forward để bỏ cls, thêm ref-conditioning. (Được gọi từ `get_model` ở trên.) rfdetr.roboflow.c...

Lưu ý ổn định huấn luyện khi bỏ cls:

- Khởi tạo decoder queries từ checkpoint sẵn có; freeze backbone + encoder 5–10 epoch đầu để regression không nổ.
- Dùng λ : $L1=5.0$, $GIoU=2.0$ làm mặc định hợp lý; top-K queries=100.
- Với pseudo từ similarity, áp dụng EMA pseudo và strong/weak augmentation consistency để giảm nhiễu.

Bạn có thể xác nhận sơ đồ gọi hàm và lớp ở tài liệu chính thức (RFDETR → `get_model` → `Model`, và `train/predict` của RFDETR) trước khi áp patch. rfdetr.roboflow.c... +1

Sources

Nói kỹ hơn về phương án 2, soi code detr trong repo để giải thích cho tôi nên thay chỗ nào

Kế hoạch "reference-conditioned queries" — trở đúng chỗ cần sửa trong repo và cách vá tối thiểu.

Điểm móc trong repo

- `rf detr/detr.py`
 - `RFDETR.get_model(config)` tạo `Model(**config.dict())`. Đây là lối vào để thêm tham số cấu hình mới (ví dụ `ref_conditioning`, `use_box_only`) và để bạn truyền `ref-image` vào `train/predict`. Roboflow
 - `RFDETR.predict(...)`: vòng tiền xử lý ảnh, nơi bạn có thể mở rộng API để nhận thêm `ref_images` đi kèm mỗi frame. Roboflow
- Phần “đầu ra phân loại” nằm trong mô hình con (head) — upstream issues chỉ rõ đầu phân loại là `class_embed` trong `rf detr/models/lwdetr.py`. Bạn sẽ không đụng vào decoder cốt lõi, chỉ thay cách khởi tạo query và có thể bypass `class_embed`. GitHub

Sửa 1 — mở API để đưa reference vào pipeline

Mục tiêu: thêm ref vào `train/predict` và đẩy xuống mô hình.

1. Mở rộng API:

- Trong `rf detr/detr.py`, đổi chữ ký:
 - `RFDETR.predict(images, ..., ref_images=None, **kwargs)` — giữ tương thích, nếu `ref_images` is `None` dùng queries học được như cũ. Bạn đặt ref theo từng ảnh: list cùng độ dài với `images`. Roboflow
 - Tương tự trong `train(...)` → khi build `DataLoader`, batch trả về `(ref_img, img, targets)`.

2. Tiền xử lý:

- Ở loop chuẩn hoá trong `predict(...)`, hiện repo convert ảnh về `torch.Tensor` [0..1], shape `(C,H,W)`. Làm y hệt cho `ref_img`. Giữ nguyên kiểm tra kênh/chuẩn hoá. Đưa cả hai xuống `self.model.model` qua kwargs mới, ví dụ `self.model.model(ref_img=ref_tensor, img=img_tensor, ...)`. Roboflow

Sửa 2 — thêm “reference-conditioned queries” trong mô hình

Mục tiêu: thay cách tạo `query_embed` để phụ thuộc ref, không đụng vào backbone/encoder/decoder gốc.

Vị trí thay:

- File mô hình lõi: `rf detr/models/lwdetr.py` (class `LWDETR`). Tại đây có:
 - `self.class_embed = nn.Linear(hidden_dim, num_classes+1)` (đầu phân loại) và `self.bbox_embed` (đầu hồi quy hộp).

- `self.query_embed = nn.Embedding(num_queries, hidden_dim)` để khởi tạo queries.
- Hàm `forward(...)` gọi backbone→encoder→decoder với `query_embed`. (Đường dẫn và biến danh định y như upstream issue mô tả). [GitHub](#)

Các bước và tối thiểu:

1. Adapter cho reference

Thêm các module sau trong `__init__`:

python

```
# giả sử có self.hidden_dim và self.backbone đã sẵn
self.ref_pool = torch.nn.AdaptiveAvgPool2d((1,1))
self.ref_proj = torch.nn.Sequential(
    torch.nn.Linear(self.backbone_out_channels, self.hidden_dim),
    torch.nn.LayerNorm(self.hidden_dim),
)
# tùy chọn: cross-attn nhẹ trước decoder để “truyền” ref vào queries
self.ref_cross_attn = torch.nn.MultiheadAttention(
    embed_dim=self.hidden_dim, num_heads=8, batch_first=True
)
```

2. Tạo ref-queries

Trong `forward`, nhận thêm `ref_img: Tensor | None`:

python

```
def forward(self, img, *, ref_img=None, targets=None, **kwargs):
    # features cho frame
    feats, pos, mask = self.backbone(img) # y như trước

    # queries học sẵn (giữ để tương thích checkpoint)
    q_learned = self.query_embed.weight.unsqueeze(0).repeat(img.size(0), 1, 1)
    # [B, Q, D]

    if ref_img is not None:
        # encode ref dùng backbone dùng chung để chia sẻ biểu diễn
        ref_feats, _, _ = self.backbone(ref_img)
        ref_map = ref_feats[-1] # Lấy stage cuối [B, C, H, W]
        ref_vec = self.ref_pool(ref_map).flatten(1) # [B, C]
        q_ref = self.ref_proj(ref_vec).unsqueeze(1) # [B, 1, D]
        # trộn: cho q_learned “ngửi” ref một lượt cross-attn
        q_cond, _ = self.ref_cross_attn(q_learned, q_ref, q_ref) # [B, Q, D]
```

```

        query_embed = q_learned + q_cond
    else:
        query_embed = q_learned

    # gọi transformer decoder như cũ, chỉ thay query_embed truyền vào
    hs, references = self.transformer(
        feats, mask, pos, query_embed=query_embed
    )
    # heads: bbox như cũ, cls có thể giữ để tương thích nhưng không dùng trong
    loss

    outputs_classes = self.class_embed(hs) # hoặc bỏ qua trong loss
    outputs_coords = self.bbox_embed(hs).sigmoid()
    out = {"pred_logits": outputs_classes[:, -1], "pred_boxes":
    outputs_coords[:, -1]}
    return out

```

3. Cấu hình mô hình

- Thêm cờ cấu hình ở `ModelConfig` (đi qua `RFDETR.get_model(config)`):
`ref_conditioning: Literal["none", "gap_xattn"]="gap_xattn",`
`use_box_only: bool=True` để điều khiển loss sau này. Thêm kwargs này được chuyển nguyên vẹn vào `Model(**config.dict())`. Roboflow

Sửa 3 — vô hiệu hoá phân loại, giữ box-only

Mục tiêu: không cần nhãn lớp. Vẫn train hồi quy box.

- Không chạm vào kiến trúc decoder. Chỉ đổi loss/matcher để bỏ thành phần class.
- Trong huấn luyện, để tương thích checkpoint bạn có thể **giữ** `class_embed` **nhưng** `weight=0`.

Nơi áp dụng:

- Bộ huấn luyện/loss của RF-DETR được gọi bên trong `self.model.train(...)` từ `RFDETR.train_from_config`. Bạn không cần đổi call site; chỉ cần thêm tham số `use_box_only=True` vào `ModelConfig` và ở lớp `Model` (hoặc criterion bên trong nó) **bỏ tính** `loss_ce` và **khử** `cost_class` **trong Hungarian** (giữ `L1 + GIoU`). Upstream xác nhận head phân loại là `class_embed` — để lại nhưng không đóng góp loss.

Roboflow +1

Pseudo-patch cho criterion/matcher bên trong mô hình:

```
python
```

```

if self.use_box_only:
    # matcher
    cost_bbox = lam_l1 * box_l1_cost(pred_boxes, tgt_boxes) + lam_giou *
    giou_cost(pred_boxes, tgt_boxes)
    indices = hungarian(cost_bbox) # không có cost_class
    # losses
    loss_bbox = l1(pred_boxes[indices], tgt_boxes) + lam_giou*(1-
    giou(pred_boxes[indices], tgt_boxes))
    losses = {"loss_bbox": loss_bbox}
else:
    # nhánh cũ (cls + box)

```

Sửa 4 — dữ liệu và target

- Dataloader phải trả `ref_img` cùng `img` và **targets chỉ chứa boxes** (không `labels`). Khi `use_box_only=True`, criterion của bạn **không đọc** `labels`.
- Nếu `ref` là crop “đúng vật thể”, bạn có thể khởi động bằng pseudo-label: `tgt_box = full_image_box` cho ảnh `ref`; với frame thật, dùng Hungarian trên score decoder để chọn box tốt nhất rồi tự-nhấn dần (self-training) — phần này nằm ở dataset/loop huấn luyện chứ không đòi sửa kiến trúc.

Vì sao đây là “DETR-way” chứ không phá kiến trúc

- Bạn không đụng backbone, encoder, decoder; chỉ thay **nguồn queries** (`self.query_embed`) bằng phiên bản đã **điều kiện hoá ref**; vẫn dùng cùng call vào transformer. Đây là đúng triết lý DETR: queries chứa “ý định tìm gì”; `ref` cung cấp “ý định”.
- Các file/entry points cần đụng tới đều xác định: `rfdetr/detr.py` cho API & config → `Model(**config.dict())`; trong mô hình con (`rfdetr/models/lwdetr.py`) nơi có `class_embed` / `bbox_embed` / `query_embed` và `forward`. [Roboflow +1](#)

Gợi ý thực thi an toàn

- Giữ nguyên `class_embed` để load checkpoint, nhưng **đặt weight cls = 0** trong loss.
- Freeze backbone + encoder 5–10 epoch đầu để regression không “trôi”.
- Duy trì `num_queries` như cũ; không cần tăng khi thêm conditioning. (Có issue bàn về tăng `num_queries` → lỗi giới hạn; tránh đổi tùy khi chưa kiểm soát đầy đủ.) [GitHub](#)

Kết quả: mô hình vẫn là DETR, inference vẫn đi qua `RFDETR.predict`, nhưng nay mỗi ảnh có thể kèm `ref_img`; decoder queries đã được “tiêm” ngữ nghĩa từ `ref` để hồi quy bbox mà không cần nhấn lớp.

